

Programmation fonctionnelle 2

Preuves et programmes

CM1

Pierre Rousselin

Université Sorbonne Paris Nord

janvier 2026

Organisation du module

Programmation fonctionnelle et OCaml

Hello, OCaml

Les assistants de preuve et Rocq

`type` et `match` (OCaml)

Inductive, Definition, `match` en Rocq

Preuves par disjonctions de cas

Organisation en 2026

- ▶ Chaque semaine :
 - ▶ cours magistral le mardi de 13h45 à 15h15
 - ▶ 1h30 de travaux dirigés
 - ▶ 1h30 (8 semaines) ou 3h (4 semaines) de travaux pratiques
- ▶ vacances d'hiver (**1 seule semaine**) la semaine du 2 mars
- ▶ partiel 1 (3h, sur papier) la semaine du 9 mars
- ▶ vacances de printemps semaines du 20 et du 27 avril
- ▶ partiel 2 (3h, sur papier) la semaine du 18 mai
- ▶ seconde chance (3h, sur papier) la semaine du 22 juin

Note du module

- ▶ Évaluation continue (EvC) : plusieurs devoirs à rendre, éventuellement des TP notés
- ▶ Note à la fin du semestre :

$$N_{\text{PF2}} = \max \left(\text{Pa}_2, \frac{\text{EvC} + \text{Pa}_1 + 2\text{Pa}_2}{4} \right)$$

Note du module

- ▶ Évaluation continue (EvC) : plusieurs devoirs à rendre, éventuellement des TP notés
- ▶ Note à la fin du semestre :

$$N_{\text{PF2}} = \max \left(\text{Pa}_2, \frac{\text{EvC} + \text{Pa}_1 + 2\text{Pa}_2}{4} \right)$$

- ▶ Si $N_{\text{PF2}} < 12$, seconde chance (S) et la note est

$$N'_{\text{PF2}} = \max \left(N_{\text{PF2}}, S, \frac{\text{EvC} + 3S}{4} \right)$$

- ▶ UE fondamentale qui vaut 7 ECTS (6 en DL)

But du module

- ▶ Se perfectionner en programmation fonctionnelle en OCaml et
- ▶ écrire des preuves **formelles, sur machine** de programmes en Rocq.

Organisation du module

Programmation fonctionnelle et OCaml

Hello, OCaml

Les assistants de preuve et Rocq

type et match (OCaml)

Inductive, Definition, match en Rocq

Preuves par disjonctions de cas

Traits principaux de la programmation fonctionnelle en 2026

- ▶ Immutabilité (*immutability*, immuabilité, non mutabilité) : on manipule des *valeurs* qu'on ne peut pas modifier.
- ▶ Fonctions comme valeurs : les fonctions sont des valeurs comme les autres, une fonction peut prendre des fonctions en paramètres et/ou retourner des fonctions.
- ▶ Le plus souvent (mais pas toujours) avec un typage statique fort.
- ▶ De plus en plus de langages populaires adoptent ces traits.

La programmation fonctionnelle à travers les âges

- ▶ années 1930 : Alonzo Church développe le *lambda calcul* comme modèle mathématique de calcul (en parallèle des machines d'Alan Turing). Ce modèle de calcul repose sur l'abstraction (définition de fonction) et l'application d'une fonction à une valeur.
- ▶ Lisp (*LISt Processing*) est développé par John MacCarthy au MIT en 1958 (l'un des premiers langages haut niveau avec le Fortran, Algol et Cobol). <https://lisp-lang.org/>
Lisp n'est pas typé, mais a déjà un *Garbage Collector* !
- ▶ ML (*Meta Language*) développé par Robin Milner à l'université d'Édimbourg en 1973 (déjà pour un système de preuves formelles), avec du typage statique puis du polymorphisme (Système F de Girard et Reynolds)

La programmation fonctionnelle à travers les âges (2)

- ▶ *Caml* (au départ *Categorical Abstract Machine Language*), descendant de ML, développé par l'INRIA (Institut National de Recherche en Informatique et en Automatique) en France à partir de 1985, descendant de ML.
- ▶ Nouvelle implémentation *Caml Light* par Xavier Leroy et Damien Doligez en 1990.
- ▶ En 1996, Didier Rémy et Jérôme Vouillon ajoutent un système d'objets et le langage est renommé *OCaml* (O pour *Object*, bien qu'on s'en serve finalement assez peu).
- ▶ En parallèle, création du langage *Haskell* par des universitaires majoritairement britanniques et américains en 1990. *Haskell* est plus « pur » qu'OCaml (pas de mutabilité du tout, effets très encadrés).
- ▶ Scala (sur JVM, 2004, EPFL)
- ▶ *F#* de Microsoft Research, dérivé de OCaml
- ▶ ...

Organisation du module

Programmation fonctionnelle et OCaml

Hello, OCaml

Les assistants de preuve et Rocq

type et match (OCaml)

Inductive, Definition, match en Rocq

Preuves par disjonctions de cas

Installation sur vos machines personnelles

- ▶ Toutes les infos sont sur :
`https://ocaml.org/install#linux\_mac\_bsd`
- ▶ Et si je suis sous Windows ? J'installe Linux :
 - ▶ en dual boot (ou en enlevant Windows)
 - ▶ ou dans une virtual box
 - ▶ ou, faute de mieux, installer *Windows Subsystem for Linux*
- ▶ En bref, installer **opam** (*OCaml Package Manager*) et l'initialiser avec :

```
$ curl -fsSL https://opam.ocaml.org/install.sh | sh  
$ opam init -a
```
- ▶ Lire les remarques et messages d'erreur d'**opam**. En particulier, il peut vous demander s'il faut modifier le fichier de configuration du shell (et ajouter un répertoire au **PATH**), etc.

Utilisation au quotidien avec compilation

- ▶ Éditeur de votre choix : emacs, Vim ou neovim, kate, vscode (mais sans copilot), ...
- ▶ En général, on écrit son code source dans l'éditeur (par exemple `vim foo.ml`) et ensuite :
- ▶ on peut utiliser le compilateur natif

```
$ ocamlc foo.ml -o foo.exe
$ ./foo.exe
```
- ▶ on peut utiliser le compilateur vers un *bytecode* intermédiaire (un peu comme avec Java) :

```
$ ocamlc foo.ml -o foo.byte
$ ocamlrun foo.byte # ou directement ./foo.byte
```
- ▶ On a ici utilisé les extensions `.exe` et `.byte` pour faire la différence entre l'exécutable natif et le bytecode. Ces extensions ne sont pas nécessaires.

Utilisation au quotidien avec interprète

- ▶ (recommandé) Installer `utop` qui est un interprète OCaml plus confortable que `ocaml` :

```
$ opam install utop
```

- ▶ on peut utiliser l'interprète `ocaml` ou l'interprète `utop` directement sur le code source :

```
$ ocaml foo.ml # ou utop foo.ml
```

- ▶ on peut *charger* le code source dans l'interprète et expérimenter/tester.

```
$ utop
```

```
# #use "foo.ml";;
```

```
# (* on a accès à toutes les définitions de foo.ml *)
```

Organisation du module

Programmation fonctionnelle et OCaml

Hello, OCaml

Les assistants de preuve et Rocq

type et match (OCaml)

Inductive, Definition, match en Rocq

Preuves par disjonctions de cas

Assistant de preuve

- ▶ Programme informatique permettant la vérification de preuves mathématiques
 - ▶ pour les mathématiciens ($\nearrow \nearrow$ mais encore faible en proportion)
 - ▶ pour les propriétés de programmes informatiques (depuis de nombreuses années déjà, mais en constante progression)
- ▶ Le premier : Automath de Nicolas de Bruijn, TU Eindhoven, 1966
- ▶ Mizar, Andrzej Trybulec, Université de Varsovie, 1973
- ▶ Coq/Rocq, Thierry Coquand, Gerard Huet, Christine Paulin-Mohring, Inria, 1984
- ▶ HOL-Isabelle, Lawrence Paulson, Cambridge, Munich, 1986
- ▶ Lean, Leonardo de Moura, Microsoft Research, 2013

Le seul qui n'est plus développé est Automath, chacun a ses avantages et ses inconvénients.

Coq/Rocq

- ▶ Nommé d'après Thierry Coquand, inventeur du « Calculus of Constructions » (COC, d'où aussi Coq), une variante de la théorie des types de Per Martin-Löf qui est la base théorique de Coq.
- ▶ Renommé Rocq (comme Rocquencourt, siège historique de l'Inria) en 2025 car en anglais un homophone de « Coq » peut être considéré comme grossier.
- ▶ Quelques traits de Coq/Rocq :
 - ▶ prend le calcul très au sérieux, beaucoup de théorèmes peuvent être simplement obtenu « par calcul » ;
 - ▶ prend sa cohérence interne très au sérieux, on n'ajoute des fonctionnalités seulement lorsqu'il est prouvé (formellement ou non) qu'elles préservent les propriétés clé du système ;
 - ▶ prend la « compatibilité ascendante » très au sérieux (les développements faits avec les anciennes versions de Coq devraient fonctionner avec les versions plus récentes).
 - ▶ revers de la médaille : on doit donc composer avec certains mauvais choix « historiques » qui encombrant l'assistant.

Quelques projets spectaculaires en Rocq, côté informatique

- ▶ CompCert, un compilateur C prouvé correct en Rocq :
<https://compcert.org/>
(Xavier Leroy et collaborateurs, 2005 – aujourd’hui)
- ▶ Fiat Cryptography, un générateur de code pour la cryptographie prouvé en Coq : <https://github.com/mit-plv/fiat-crypto>
(2017 Adam Chlipala, Andres Erbsen, Jason Gross, ..., 2017 – aujourd’hui)
- ▶ *Software Foundations* :
<https://softwarefoundations.cis.upenn.edu/>
de Benjamin Pierce et autres
- ▶ why3 : un *framework* de vérification déductive de programmes :
<https://www.why3.org>

Quelques projets spectaculaires en Rocq, côté mathématiques

- ▶ La preuve formelle du théorème des 4 couleurs (44000 loc) :
`https://github.com/coq-community/fourcolor`
(2005 Georges Gonthier et Benjamin Werner)
- ▶ La preuve formelle du théorème de Feit-Thompson (40000 loc) :
`https://github.com/math-comp/odd-order`
(2012 Georges Gonthier et de nombreux collaborateurs)

Rocq en pratique pour vous

- ▶ Sur les salles TP : Coq 8.20, coqide
- ▶ Sur vos machines personnelles :
 - ▶ installer avec Opam
 - ▶ installez aussi **rocqide** ou bien un plugin pour éditeur de texte :
 - ▶ VsCoq, une extension pour VsCode,
 - ▶ CoqTail pour Vim,
 - ▶ Proof General pour Emacs.

Organisation du module

Programmation fonctionnelle et OCaml

Hello, OCaml

Les assistants de preuve et Rocq

type et **match** (OCaml)

Inductive, Definition, match en Rocq

Preuves par disjonctions de cas

variant (type union)

En OCaml on définit un type avec

- ▶ le mot-clé **type** et
- ▶ des *constructeurs*
 - ▶ qui peuvent avoir un tuple d'arguments de certains type **dont le type qui est lui-même en train d'être défini**
 - ▶ dont le nom ne commence pas par une majuscule, sauf s'il est constant
 - ▶ séparés par des barres verticales (|).

Exemples de variants OCaml

- ▶ Remarque : dans `utop`, utiliser une directive `#show foo` pour voir la définition du type `foo`.
- ▶ `type bool = false | true`
Les booléens d'OCaml sont de type `bool` et prennent les seules valeurs `true` et `false`.
- ▶ `type unit = ()`
Le type `unit` d'OCaml n'a qu'une valeur, notée `()`. Elle sert à indiquer qu'une opération a des effets de bords. Comme dans `val print_string : string -> unit`.
- ▶ On peut définir son propre variant `number`¹ :

```
type number = Float of float | Int of int | Error
let n1 = Float 2.3
let n2 = Int 0
```
- ▶ On peut considérer un type à un seul constructeur (même si en pratique, on utilise plus souvent un `record` pour ça) :

```
type point = Point of float * float
```

1. exemple tiré du manuel de référence d'OCaml

Utiliser les variants : `match`

- ▶ La structure de contrôle de base d'OCaml est le *pattern matching* sur le ou les constructeurs d'un élément d'un type.
- ▶

```
let b = true
let n = match b with false -> 42 | true -> 3
```
- ▶ Ici, `n` vaut 3 car `b` vaut `true`.
- ▶ On s'en sert souvent pour définir des fonctions.

```
let negb b =
  match b with
  | true -> false
  | false -> true
```

- ▶ Avec `point` :

```
let symmetric p =
  match p with
  Point (x, y) -> (-. x, -. y)
```


match de base

Syntaxe :

```
match value with  
| Cons1 (arg11, arg12, ..., arg1n1) -> expr1  
| Cons2 (arg21, arg22, ..., arg1n2) -> expr2  
| ...
```

où `value` est une expression d'un certain type variant `t` dont les constructeurs sont `Cons1`, `Cons2`, etc, ayant respectivement `n1`, `n2`, etc arguments (la barre verticale du premier cas est optionnelle).

match de base

Syntaxe :

```
match value with  
| Cons1 (arg11, arg12, ..., arg1n1) -> expr1  
| Cons2 (arg21, arg22, ..., arg1n2) -> expr2  
| ...
```

où **value** est une expression d'un certain type variant **t** dont les constructeurs sont **Cons1**, **Cons2**, etc, ayant respectivement **n1**, **n2**, etc arguments (la barre verticale du premier cas est optionnelle).

Sémantique :

- ▶ si **value** a été construite par le constructeur numéro **i** avec des arguments éventuels,
- ▶ les variables **argi1**, **argi2**, ..., **argini** sont liées à ces arguments dans **expr_i** correspondante,
- ▶ qui est elle-même évaluée et est la valeur de toute l'expression **match**.

match de base

Remarques :

- ▶ Le compilateur vous force à prendre en compte tous les cas.
- ▶ Un argument non utilisé peut (bonne pratique) être remplacé par un blanc souligné (_).
- ▶ Des cas peuvent être regroupés.

```
let is_float n =  
  match n with  
  | Int _ | Error -> false  
  | Float _ -> true
```

- ▶ On peut aussi regrouper tous les cas restants avec un souligné :

```
let is_float n =  
  match n with  
  | Float _ -> true  
  | _ -> false
```

Types primitifs et `match`

- En fait, `int`, `float` et `char` peuvent aussi être vus comme des variants avec un nombre fini (mais grand, voire très grand) de constructeurs, ce qui autorise à utiliser `match` sur ces types.

```
let is_42_or_12 n =  
  match n with  
  | 42 | 12 -> true  
  | _ -> false
```

- Pour `bool`, « en vrai » c'est primitif en OCaml pour des raisons de performances et plutôt que

```
match b with true -> expr1 | false -> expr2
```

on va préférer

```
if b then expr1 else expr2
```

Sucre syntaxique

- C'est tellement fréquent pour définir des fonctions qu'on peut utiliser le mot-clé **function** :

```
let is_float = function  
| Float _ -> true  
| _ -> false
```

qui remplace

```
let is_float = fun n ->  
  match n with  
  | Float _ -> true  
  | _ -> false
```

- Pour les types à un seul constructeur (comme **point** ci-dessus), on peut écrire

```
let fst (Point (x, _)) = x
```

C'est un *motif irréfutable*, toujours vrai pour un point qui n'a qu'un constructeur.

Organisation du module

Programmation fonctionnelle et OCaml

Hello, OCaml

Les assistants de preuve et Rocq

`type` et `match` (OCaml)

Inductive, Definition, match en Rocq

Preuves par disjonctions de cas

Exemple avec les booléens de Rocq

```
Inductive bool = true | false.  
Definition negb b :=  
  match b with  
  | true => false  
  | false => true  
  end.  
Check negb. (* negb : bool -> bool *)  
Compute (negb true). (* false *)  
Lemma negb_false : negb false = true.  
Proof. (* on entre en mode preuve *)  
  simpl. (* true = true *)  
  reflexivity. (* All goals completed. *)  
Qed. (* Rocq va vérifier que le terme est du bon type. *)
```

Fichier source Rocq

- ▶ Un fichier source Rocq est une suite de *commandes* dans un langage appelé « vernaculaire ».
- ▶ Le fichier lui-même a un nom comme, par exemple, `Bool.v` (se termine par `.v` comme vernaculaire).
- ▶ Les commandes vernaculaires permettent de :
 - ▶ ajouter des définitions : ici le type `bool`, les constructeurs `true` et `false` avec la commande `Inductive`;
 - ▶ ajouter une définition de fonction avec la commande `Definition`;
 - ▶ demander le type d'une expression avec `Check`, évaluer une expression avec `Compute`.
- ▶ L'ensemble des définitions (de types, de fonctions, de constantes, ...) s'appelle **l'environnement global courant**.
- ▶ On peut dire que les commandes vernaculaires permettent de manipuler l'environnement.
- ▶ Syntaxe : les commandes vernaculaires commencent par une majuscule et se terminent toujours par un point.

Inductive

Les exemples vus précédemment en OCaml deviennent :

```
From Coq Require Import Reals BinInt.
```

```
Inductive bool := true | false.
```

```
Inductive unit := tt. (* ici c'est tt à la place de () *)
```

```
(* pour number, on va, par simplicité, éviter les types int et float et  
    utiliser les nombres réels et les entiers relatifs *)
```

```
Inductive number :=
```

```
| Real (r : R)
```

```
| Int (i : Z)
```

```
| Error.
```

```
Inductive point := Point (x y : R).
```

```
Definition symmetric (p : point) : point :=
```

```
  match p with
```

```
  | Point x y => Point (- x) (- y)
```

```
  end.
```

```
Definition is_real (n : number) : bool :=
```

```
  match n with
```

```
  | Real r => true
```

```
  | Int i => false
```

```
  | Error => false
```

```
  end.
```

Sucre syntaxique

Comme en OCaml, on peut regrouper des cas et utiliser le souligné pour signifier qu'un argument de constructeur n'est pas utilisé.

```
Inductive number :=  
  | Float (f : float)  
  | Int (i : int)  
  | Error.
```

```
Definition is_real' (n : number) : bool :=  
  match n with  
  | Int _ | Error => false  
  | Real r => true  
  end.
```

```
Definition is_real'' (n : number) : bool :=  
  match n with Real _ => true | _ => false end.
```

Motif irréfutable et `if`

Comme en OCaml, dans le cas d'un seul constructeur, on peut utiliser un motif irréfutable directement :

```
Definition symmetric '(Point x y) :=  
  Point (- x) (- y).
```

Pour le type `bool`, on a aussi une notation `if then else`.

```
Definition andb (b1 b2 : bool) : bool :=  
  if b1 then b2 else false.
```

Jeu des 7 différences

Différences avec OCaml :

- ▶ les noms des constructeurs non constants d'un **Inductive** n'ont pas besoin de commencer par une majuscule
- ▶ les constructeurs sont des vraies fonctions et peuvent avoir plusieurs arguments (au lieu d'un tuple d'arguments en OCaml)
- ▶ la flèche du **match** est la flèche $\Rightarrow$ (et pas $\rightarrow$)
- ▶ **match** se termine par **end**
- ▶ les définitions se font avec $:=$
- ▶ les motifs irréfutables s'écrivent avec une apostrophe (et pas directement)

Organisation du module

Programmation fonctionnelle et OCaml

Hello, OCaml

Les assistants de preuve et Rocq

`type` et `match` (OCaml)

Inductive, Definition, `match` en Rocq

Preuves par disjonctions de cas

Lemmes et preuves

- ▶ Coq permet également d'énoncer des théorèmes avec les commandes **Lemma**, **Theorem**, **Proposition**, ...
- ▶ On utilisera tout le temps **Lemma** par soucis de cohérence.
- ▶ Ensuite, il faut apporter une preuve. Ceci se fait entre **Proof**. et **Qed**.
- ▶ Dans une preuve, on fait évoluer l'état de la preuve avec des commandes appelées *tactiques*.
- ▶ Pour le moment, voyons :
 - ▶ **destruct** qui est l'équivalent de **match** en mode preuve
 - ▶ **simpl** qui demande à Rocq d'évaluer (calculer) ce qu'il peut dans le but en utilisant les définitions.
 - ▶ **reflexivity** qui termine une preuve d'égalité lorsque le but a la forme $a = a$.

Exemple avec bool

Lemma andb_true_r (b : bool) : andb b true = b.

Proof.

destruct b.

- (* b = true, le but est andb true true = true *)

simpl. (* replace andb true true par true *)

reflexivity.

- (* b = false, le but est andb false true = false *)

simpl. (* replace andb false true par false *)

reflexivity.

Qed.

Exemple avec point

```
Lemma symmetric_symmetric (p : point) :
```

```
  symmetric (symmetric p) = p.
```

```
Proof.
```

```
  destruct p as [x y].
```

```
  simpl.
```

```
  rewrite !Ropp_involutive. (* plus sur rewrite plus tard *)
```

```
  reflexivity.
```

```
Qed.
```


Exemple avec number

```
Definition opp (n : number) :=  
  match n with  
  | Real r => Real (- r)  
  | Int i => Int (- i)  
  | Error => Error  
end.
```

```
Lemma opp_opp (n : number) : opp (opp n) = n.
```

```
Proof.
```

```
destruct n as [r | k |].  
- (* 1er cas : n = Real r *)  
  simpl.  
  rewrite Ropp_involutive.  
  reflexivity.  
- (* 2ème cas : n = Int k *)  
  simpl.  
  rewrite Z.opp_involutive.  
  reflexivity.  
- (* 3ème cas : Error *)  
  simpl.  
  reflexivity.
```

```
Qed.
```

destruct

- ▶ Si x est une variable d'un type inductif à k constructeurs $\text{Cons}_1, \dots, \text{Cons}_k$:

```
destruct x as [v11 ... v1n1 | v21 ... v2n2 | ... | vk1 ... vknk].  
- (* cas 1 : x est remplacé par (Cons1 v11 ... v1n1) *)  
- ...  
- (* cas k : x est remplacé par (Cons k vk1 ... vknk) *)
```

- ▶ Pour un constructeur sans argument, la partie correspondante dans les crochets est laissée vide, pour `bool`, on aurait pu écrire :

```
destruct b as [].
```

- ▶ Bonne pratique : si on utilise un nom, on nomme nous-même.
- ▶ Bonne pratique : utiliser les *bullets* (ici `-`, mais `+`, et `*` sont possibles) pour *se concentrer* sur une seule preuve (*strict focussing*).
- ▶ Parfois, on veut garder x :
 - ▶ ajouter `eqn:nomEquation` à la fin de la tactique pour faire apparaître l'égalité `nomEquation : x = Cons i vi1 ... vini` dans chaque cas.